AURORA: Attention-intended Dual Verification with Adaptive Rollback for Large Language Model Agents

Ziqi Wang[1], Huaiyuan Yao[2], Hua Wei[2], Hang Zhou[3*], and Xiao Luo[4]

***Abstract*— This paper investigates the problem of large language model (LLM) agents for environment-attended problem solving, which aims to integrate the reasoning ability into the interactions in dynamic environments. Existing works typically enhance alternative reasoning and acting with auxiliary assistant models to mitigate the potential biases and errors. However, these methods could introduce new errors or redundancy due to inaccurate judgments of current states and actions, leading to inferior performance. In this paper, we propose a new framework named Attention-intended Dual Verification with Adaptive Rollback (AURORA) to enhance the reliability of LLM agents. The core of our AURORA is to leverage the attention map derived from trajectory texts to enable accurate rollback mechanism from two perspectives. In particular, our AURORA first generates interaction trajectories by environment exploration. More importantly, we generate two learnable validation scores by analyzing the attention masks over trajectory texts, indicating physical validity and overall coherence, respectively. By comparing these scores with the predefined thresholds, we adaptively determine whether to invoke next action proposals and the potential rollback depth, which guarantee the reliability of our decision. Extensive experiments across multiple benchmarks validate the effectiveness of the proposed AURORA. The code is available at https://github.com/ziqiwang0908/AURORA.**

I. INTRODUCTION

LLM-powered agents have shown strong performance on complex, long-horizon tasks [1], particularly in embodied tasks [2] and robotic task planning [3]. However, such tasks require dozens of sequential actions [4], where a single erroneous step can propagate and cause task failure, and restarting from scratch will discard all valid progress [5]. Rollback addresses this by restoring execution to a previous checkpoint when an error is detected, preserving the valid steps and only undoing the faulty portion. Nevertheless, the generator model tends to be biased toward its own outputs and cannot reliably judge whether its own steps are correct [6]. To address this, existing frameworks decouple execution from supervision by introducing a separate assistant model that reviews trajectory after each step and decides whether to trigger a rollback [7].

This unconditional triggering strategy, however, introduces a counterintuitive failure mode: the more aggressively the assistant intervenes, the more noise is compounded into the trajectory [8], [9], [10]. A study have supported this in Figure 1. To solve this, a remedy is to gate intervention on the physical validity of the current action. Yet we find single-head gating insufficient: trajectory failures fall into two different modes. Local errors arise when the current action is physically invalid and can be detected step-locally; global deviations arise when the action is valid but the overall trajectory has lost coherence with the task goal, requiring reasoning over the full history, which is a distinction that a single-head gating cannot capture. This raises a question:

How can we design structured verification signals that distinguish local errors from global deviations, enabling precise and selective intervention?

This verification problem has a formulation in sequence modeling [11], [12]: trajectory texts are composed of token sequences, where the physical validity of the current step depends on its relationship with the current environment feedback tokens [13], [8], and the overall coherence of the trajectory depends on its relationship with the full history of tokens [14], [15]. The attention mechanism, by controlling which tokens participate in computation, can express both types of relationships, and is more lightweight than training separate local and global models [16], [17], [18]. This motivates our proposal of AURORA (Attention-intended Dual Verification with Adaptive Rollback), a lightweight dual-head verifier built on attention masks over trajectory texts. Unlike two independent verifiers, AURORA operates within a single shared encoder, using different attention masks to control each head's information scope: the local head attends only to the current step to assess physical validity; the global head attends to the full trajectory history, assessing the overall coherence of the accumulated trajectory with the task goal. Together, these scores form a two-dimensional verification vector. By comparing them against predefined thresholds, AURORA adaptively determines whether to invoke the next action proposal. AURORA is trained via an automated annotation pipeline using a closed-source LLM, which generates local and global labels for each trajectory step and distills them into a lightweight classifier. However, when both heads share the same attention computation, the local head's representation inevitably absorbs trajectory history, causing the two scores to converge rather than diverge. We then refer to this default shared-attention design as AURORA-Integration, and further introduce AURORA-Isolation as a refinement that masks the local head's input prior to attention to achieve true context isolation, with a capacity-dependent trade-off: larger models benefit from strict separation, while smaller ones may not. Our contributions are as follows:

❶ *New perspective on trajectory verification*: We identify

[1]Rutgers University, USA, ziqi.wang0908@rutgers.edu
[2]Arizona State University, USA, {huaiyuan, hua.wei}@asu.edu
[3]University of North Carolina at Chapel Hill, USA, hzh@unc.edu
[4]University of Wisconsin–Madison, USA, xiao.luo@wisc.edu
*Corresponding author.

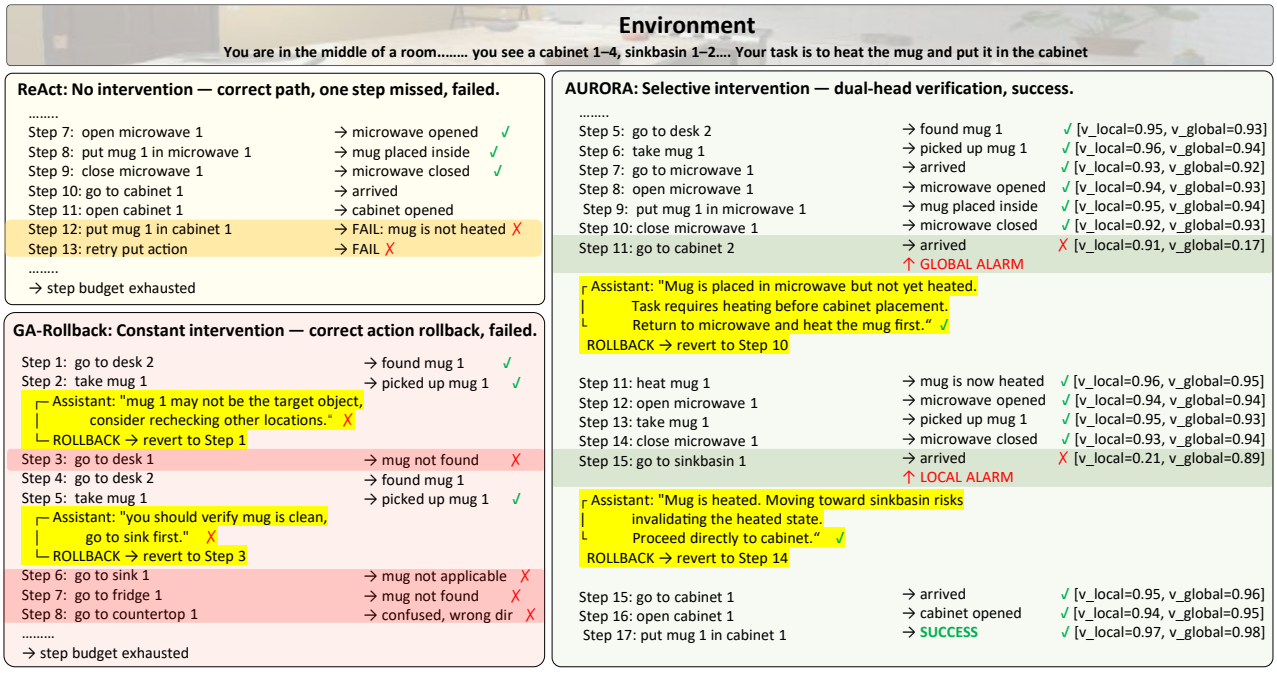


Fig. 1. Comparison of three intervention strategies on the same task. (Left Top) **ReAct**: No intervention; the agent follows a correct path but skips the heating step, leading to irreversible failure. (Left Bottom) **GA-Rollback**:Unconditional intervention; the agent correctly locates the target object in the first two steps, but the assistant misjudges this valid progress as an error and triggers an unnecessary rollback, derailing the trajectory and exhausting the step budget. (Right) **AURORA (Ours)**: Selective dual-head verification with threshold $\tau = 0.5$ for both heads. The local and global heads operate independently and complement each other, enabling precise intervention only when needed.

two distinct failure modes in agent trajectories: physical validity errors and overall coherence breakdowns, and show that single-scalar verification signals are insufficient to distinguish between them, providing direct motivation for structured dual-head verification.

❷ *Attention-mask-based dual-head verifier*: We propose AURORA, which produces physical validity and overall coherence verification signals from a single shared encoder via attention masking, without the overhead of two separate models. Training requires no manual labeling, relying on an automated annotation and distillation pipeline. We further introduce AURORA-Isolation as a technical refinement that achieves true context isolation by masking prior to attention, with capacity-dependent benefits.

❸ *Empirical evaluation*: Experiments across four models, from 3B to 14B, on the mathematical reasoning benchmark Game of 24 and the embodied environment ALFWorld, show that AURORA is **(1) effective**: AURORA-Integration achieves up to +19.2 percentage points improvement in success rate over GA-Rollback baseline; **(2) selective**: it reduces rollback frequency in 7 of 8 model-dataset combinations compared to unconditional rollback method; **(3) scalable**: improvements are consistent across all four tested model scales.

II. Related Work

A. LLM Agents

LLMs have been widely applied to building autonomous agents, leveraging their reasoning and instruction-following capabilities [19], [20], [21], [22], [23]. Structured reasoning frameworks such as Chain-of-Thought [24] and ReAct [4] guide models to generate intermediate reasoning steps before executing actions, substantially improving agent performance on complex tasks. However, in long-horizon decision-making tasks [2], [25], such purely forward generation paradigms face fundamental limitations [26], [27]. The inherent hallucination tendencies of LLMs [28], combined with the lack of effective evaluation of intermediate steps, allow local errors to propagate along the trajectory and trigger cascading failures [6], [29], [30]. This paper addresses this limitation by introducing a step-level verification mechanism into the agent execution loop, detecting and correcting errors before they propagate along the trajectory.

B. Self-Correction and Rollback

In the context of LLM-based agents, self-correction has emerged as an important approach to improving system reliability [31], [32]. Early methods rely on the model itself to generate corrective feedback [33], [34], while subsequent work has shown that leveraging external tools or environmental observations can substantially improve correction quality [5], [35], [36]. However, reliable feedback generation has been identified as the core bottleneck of self-correction [37], [38], [39], motivating recent frameworks to introduce independent assistant models dedicated to trajectory review and feedback generation [40], [41], [42]. The agent in such correction mechanisms needs to complete or fail the full trajectory before reflecting and retrying [43], [44]. To avoid the prohibitive cost of restarting, rollback-based intervention mechanisms have been widely adopted in complex reasoning and embodied settings: the system can truncate erroneous states mid-trajectory and restore execution to a prior safe

checkpoint [25], [45], [15]. GA-Rollback [7] incorporates this mechanism into a generator-assistant framework, where the assistant evaluates each step and determines whether to trigger a rollback. However, such systems invoke the assistant unconditionally at every step, leading to low efficiency. This paper replaces this strategy with structured step-level verification signals, enabling selective and precise intervention.

C. Verification Signals and Gated Intervention

Process-level verification aims to evaluate intermediate steps of agent trajectories rather than relying solely on final outcome judgments [46], [47]. Process reward models assign fine-grained scores to individual reasoning steps to guide search and error correction, enabling earlier error localization compared to outcome-based feedback [48], [49], [50]. In embodied settings like SayCan [13], introduces affordance functions to score the physical feasibility of each candidate action, representing an early form of local step-level verification. Within agent gating frameworks [8], [51], [9], [10], [52], LATS [14] incorporates an LM-driven value function that generates a single scalar score by prompting the model to reflect on the execution trajectory to evaluate state progress and guide tree search backpropagation. Yet all of the above methods compress trajectory features into a single scalar output, unable to distinguish between local validity errors and global goal deviations[10]. This paper addresses this gap by deriving two verification scores, physical validity and overall coherence, from a single shared encoder via attention masking[18], enabling precise and selective intervention.

III. Methodology

AURORA consists of three components: a generator $\mathcal{G}$, a dual-head verifier $\mathcal{V}$, and an assistant $\mathcal{A}$. At each step, $\mathcal{V}$ produces two verification scores from attention patterns over the trajectory; if either score falls below its threshold, $\mathcal{A}$ is invoked to locate a rollback checkpoint [7]. We next define the problem setting (Section III-A), describe the verifier architecture (Section III-C), and detail the two masking variants (Section III-D).

A. Problem Formulation

Let $\mathcal{I}$ denote the task description. The agent's execution trajectory in environment $\mathcal{E}$ is defined as $\mathcal{T}t = (\mathcal{I},\ a_1, s_1,\ \ldots,\ a_t, s_t)$, where a_t is the action executed at step t and s_t is the resulting environment state. The system consists of three components: a generator $\mathcal{G}$ that produces actions and interacts with the environment; a verifier $\mathcal{V}$ that evaluates the current trajectory after each step [41], [47]; and an assistant $\mathcal{A}$ that is invoked when correction is needed, responsible for trajectory analysis and rollback. Our goal is to design $\mathcal{V}$ to output a structured two-dimensional verification signal $\mathbf{v}_t = [v_{\text{local}},\ v_{\text{global}}]^\top$ at each step, measuring the physical validity of the current action and the overall coherence of the full trajectory with the task goal, so that the system can decide whether to invoke $\mathcal{A}$ and at what rollback depth based on the type of error detected.

B. Framework Overview

The overall execution flow of the AURORA is illustrated in Figure 2. At each step, $\mathcal{G}$ generates action a_t conditioned on $\mathcal{T}_{t-1}$, interacts with the environment to obtain state s_t, and updates the trajectory to $\mathcal{T}_t$. $\mathcal{V}$ then applies different attention masking strategies over the trajectory tokens: the local head attends only to the current-step tokens to assess physical validity, while the global head attends to the full trajectory context to evaluate overall coherence, producing $\mathbf{v}_t$. If any component of $\mathbf{v}_t$ falls below its threshold, $\mathcal{A}$ is invoked [8], [51], [10]: it receives the current trajectory and the triggering source as input, identifies a rollback checkpoint k, and reverts the execution state to s_k, from which $\mathcal{G}$ resumes. Otherwise, $\mathcal{G}$ proceeds directly and $\mathcal{A}$ is not called. The complete inference and rollback procedure is formalized in Algorithm 1.

Algorithm 1 AURORA

Require: Generator G, Verifier V, Assistant A, thresholds τ_{local}, τ_{global}, max steps $T_{\max}$
Ensure: Task result (*success* or *failure*)

```
Initialize trajectory T_0 ← ∅
for t = 1, 2, ... do
    a_t ← G(T_{t-1})                                   ▷ Generate action
    s_t ← Env(a_t)                                     ▷ Execute and observe state
    T_t ← T_{t-1} ∪ {a_t, s_t}
    v_t ← V(T_t)                                       ▷ Attention-based dual-head verification
    if v_local < τ_local or v_global < τ_global then
        trigger ← {h ∈ {local, global} | v_h < τ_h}
        k ← A(T_t, trigger)                            ▷ Locate rollback checkpoint
        T_t ← T_k                                      ▷ Execute rollback
    end if
    if task complete then return success
    end if
    if t > T_max then return failure
    end if
end for
```

C. Attention-based Dual-Head Verifier

The AURORA verifier $\mathcal{V}$ comprises two heads implemented within a single shared encoder, each evaluating the trajectory from a different perspective via attention masking, as illustrated in Figure 3. The local head assesses the physical validity of the current step:

$$v_{\text{local}} = f_{\text{local}}(s_{t-1},\ a_t,\ s_t) \in [0, 1],$$

where s_{t-1} is the state before the action, a_t is the executed action, and s_t is the resulting state. This head judges whether a_t is executable and valid in the current environment context [13]. It is implemented by applying a mask that restricts attention to the current-step tokens only, so that the encoder representation is computed purely from local information without influence from the historical trajectory. The global

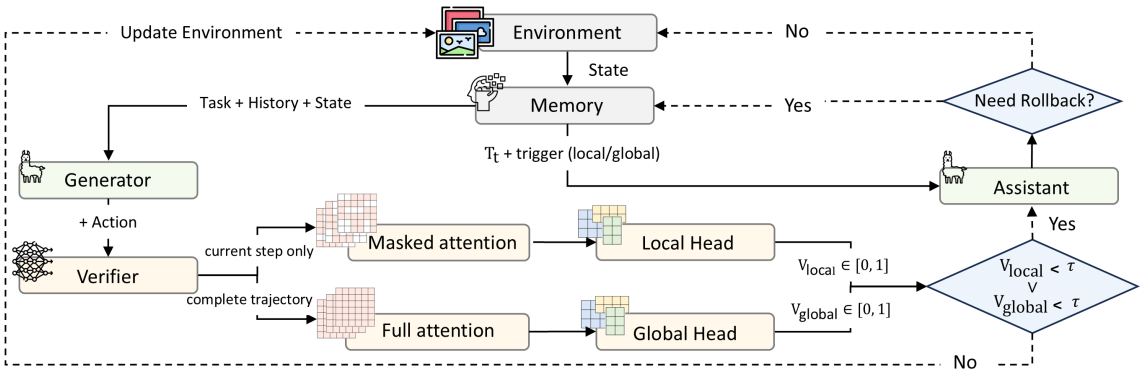


Fig. 2. **Overall execution framework of AURORA.** The verifier applies two distinct attention masking strategies over trajectory tokens: the local head uses masked attention restricted to the current step only, while the global head uses full attention over the complete trajectory, producing v_{local} and v_{global} respectively. When either score falls below its threshold τ, the assistant is selectively invoked with the triggering source (local/global) to locate a rollback checkpoint; otherwise, execution proceeds uninterrupted.

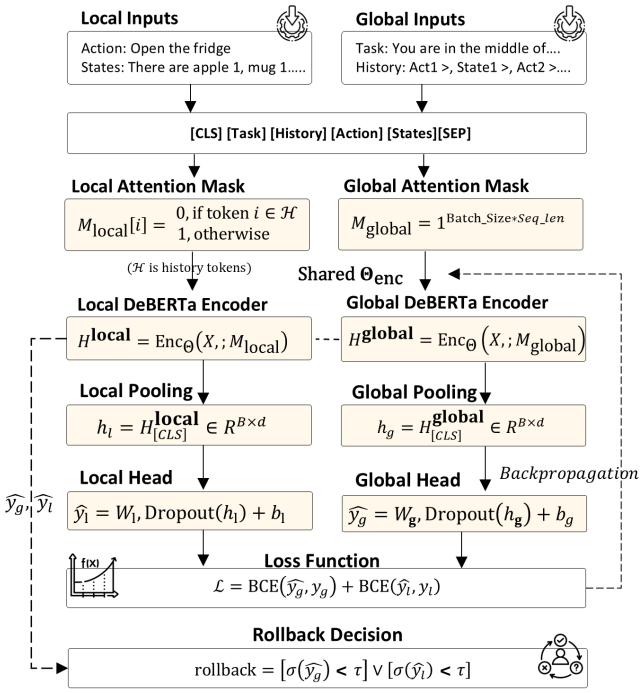


Fig. 3. **Architecture of the AURORA dual-head verifier.** A shared DeBERTa encoder processes the trajectory input, comprising task description, action history, current action, and environment feedback, under two different attention masks. The local head attends only to current-step tokens, producing a physical validity score; the global head attends to the full trajectory history, producing an overall coherence score. At inference, a rollback is triggered when either score is below its predefined threshold τ.

head evaluates whether the full trajectory is still aligned with the task goal:

$$v_{\text{global}} = f_{\text{global}}(\mathcal{T}_t) \in [0, 1],$$

taking the complete trajectory $\mathcal{T}_t$ as input to assess whether the overall execution path is still directed toward $\mathcal{I}$, capturing cross-step deviation patterns that are not visible at the single-step level. This head applies no masking, allowing full attention over all trajectory tokens to capture long-range dependencies across steps [45], [15], [14]. Both heads share the same encoder parameters and differ only in their attention mask configuration. The two outputs are combined into a verification vector $\mathbf{v}_t = [v_{\text{local}},\ v_{\text{global}}]^\top$. The system uses OR logic to decide whether to invoke $\mathcal{A}$:

$$\text{invoke}(\mathcal{A}) = \mathbf{1}\big[v_{\text{local}} < \tau_{\text{local}} \ \vee \ v_{\text{global}} < \tau_{\text{global}}\big].$$

The two signals are independent, and either one falling below its threshold is sufficient to trigger $\mathcal{A}$. The triggering source is passed to $\mathcal{A}$ as a focus hint, directing its analysis toward the corresponding error type. Under the constraint of maximum rollback depth $\overline{N}$, $\mathcal{A}$ identifies the rollback checkpoint k.

D. Attention Masking Variants: Integration and Isolation

We propose two implementations of AURORA, differing in the position at which the mask is applied relative to the attention computation [16]. **AURORA-Integration** applies masking at the attention-weight level [53]. Both heads receive the same full input sequence and share a single forward pass. The local head's mask zeroes out history token attention weights after the score computation, and its representation is obtained by mean-pooling over the current-step tokens:

$$h_l = \text{mean}(H[:,\ p_s : L,\ :]) \in R^d,$$

where p_s is the start position of the current-step tokens and L is the sequence length. The global head extracts the [CLS] token representation:

$$h_g = H[:,\ 0,\ :] \in R^d.$$

AURORA-Isolation applies masking at the input level [18], [17]. History tokens are excluded from the local head's input before entering the attention layer, so the encoder computes representations purely from current-step information. Its output is obtained via [CLS] pooling:

$$h_l = H^{\text{local}}_{[\text{CLS}]} \in R^{B\times d}.$$

The global head receives the full trajectory without masking, and its representation is similarly extracted:

$$h_g = H^{\text{global}}_{[\text{CLS}]} \in R^{B\times d}.$$

Both variants share the same encoder parameters Θ_{enc} and training pipeline; the only difference is the position at which

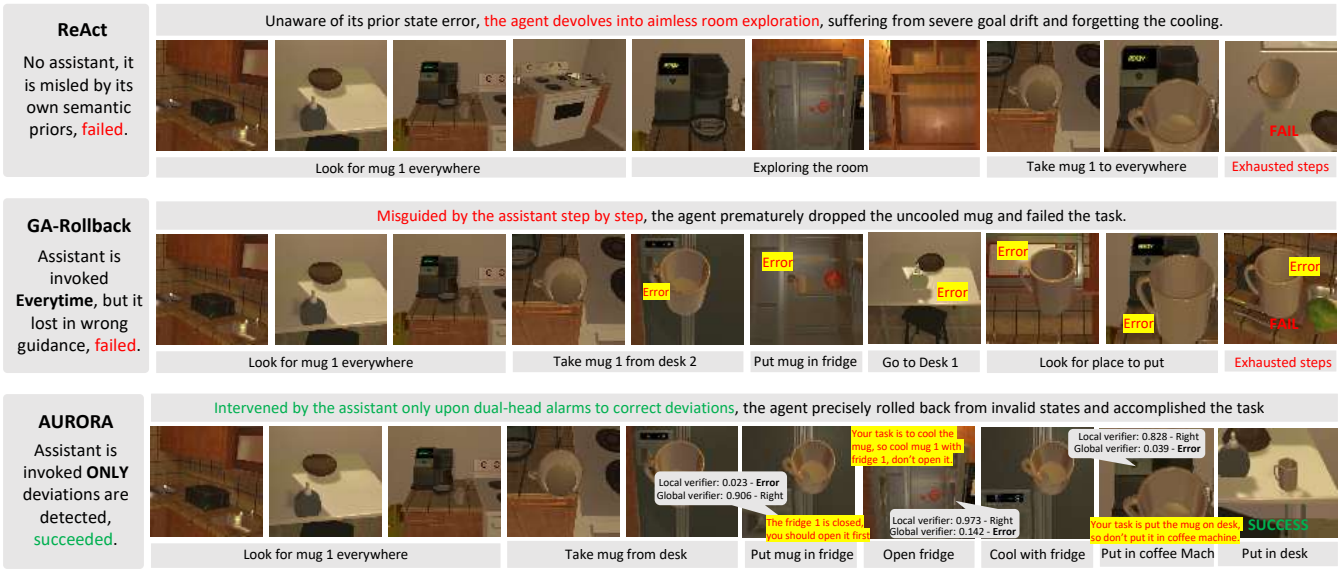


Fig. 4. **Case study on an ALFWorld cooling task.** The agent must cool a mug and place it on a desk in a room with multiple cabinets, drawers, a fridge, and desks. Each row shows one execution paradigm: *ReAct* (top) operates without any verification; *GA-Rollback* (middle) invokes the assistant unconditionally at every step; AURORA (bottom) selectively triggers the assistant based on dual-head verification signals.

the attention mask is applied. The detailed architecture is shown in Figure 3.

Together, these components form the attention-intended design of AURORA: rather than relying on separate models to enforce information boundaries, AURORA controls what each verification head can attend to through attention masking, deriving two complementary signals from a single shared encoder. AURORA-Integration and AURORA-Isolation represent two instantiations of this principle, differing only in where the mask is applied relative to the attention computation. We evaluate both variants in the following section.

IV. Experiment

A. Experimental Setup

Datasets and Evaluation. We evaluate AURORA on two benchmarks representing complementary agent settings: a symbolic reasoning task and an embodied sequential decision-making task, together covering both physical validity failures and overall coherence breakdowns that AURORA is designed to detect. Game of 24 [25] is a mathematical reasoning task where the agent must combine four numbers with arithmetic operations to reach 24. We randomly sample 500 instances and report success rate (SR). ALFWorld [2] is an embodied household task benchmark where the agent completes multi-step goal-directed tasks in a text-based simulated environment, directly reflecting the long-horizon sequential decision-making challenges central to robotic task planning. We use the official out-of-distribution test set for evaluation. In addition to SR, we report average steps to success and average rollback count on successful cases in Section IV-D, measuring judgment accuracy and execution efficiency of AURORA.

Baselines. We compare AURORA with the following methods: (1) Act-only [1], which generates actions from observations without reasoning or rollback; (2) ReAct [4], which interleaves chain-of-thought reasoning with actions; (3) Reflexion [5], which refines subsequent trajectories through multi-trial self-reflection. Results for these three methods are taken directly from GA-Rollback under identical experimental configurations. (4) GA-Rollback [7], a rollback framework without any verifier, serves as the direct baseline for AURORA. To ablate the dual-head design, we further include: (5) AURORA-Integration and (6) AURORA-Isolation, two full dual-head implementations both produce local and global verification scores but differ in context sharing strategy.

Implementation Details. We use four open-source generator models spanning 3B to 14B: Qwen2.5-3B/14B-Instruct [54], LLaMA-3.1-8B-Instruct [55], and GLM4-9B-Chat [56]. Following GA-Rollback [7], the generator and assistant share the same model. The verifier is DeBERTa-v3-base [57] fine-tuned as a dual-head binary classifier, with training labels from DeepSeek V3 [58], [59]. Manual evaluation on 200 instances confirms annotation reliability (94% agreement, Cohen's $\kappa = 0.802$). Each verifier is trained with AdamW (lr = 2e-5, weight decay = 0.01) for 5 epochs, linear warmup over the first 10% of steps, and gradient clipping at norm 1.0. Thresholds are set to $\tau_{\text{local}} = \tau_{\text{global}} = 0.5$, with validation on $\tau \in \{0.3, 0.7\}$ confirming it as a robust default. All experiments run on NVIDIA A100 and RTX 5090 GPUs.

B. Performance Comparison

Table I reports the success rates of all methods on Game of 24 and ALFWorld. We make the following observations. **Obs.❶** *Structured verification of AURORA consistently improves over unconditional rollback.* AURORA-Integration outperforms GA-Rollback across all four models on both benchmarks. On Game of 24, the relative improvement ranges of AURORA, compared with other method, are from 30% to 83%. This confirms that gating the assistant with a learned verification signal is more precise than triggering it unconditionally. The larger relative gains on Game of 24

TABLE I

SUCCESS RATE (%) ON GAME OF 24 AND ALFWORLD. **BOLD**: BEST PER COLUMN; UNDERLINE: SECOND BEST. ↑/↓: GAIN/DROP RELATIVE TO THE ACT-ONLY.

Method	Game of 24				ALFWorld			
	Qwen2.5-3B	LLaMA-3.1-8B	GLM4-9B	Qwen2.5-14B	Qwen2.5-3B	LLaMA-3.1-8B	GLM4-9B	Qwen2.5-14B
Act-only	1.0	0.4	0.0	5.4	26.1	33.6	73.1	78.3
ReAct	1.6$_{\uparrow 0.6}$	0.4	0.2$_{\uparrow 0.2}$	7.6$_{\uparrow 2.2}$	44.0$_{\uparrow 17.9}$	22.4$_{\downarrow 11.2}$	76.8$_{\uparrow 3.7}$	89.6$_{\uparrow 11.3}$
Reflexion	**5.6**$_{\uparrow 4.6}$	5.2$_{\uparrow 4.8}$	**7.2**$_{\uparrow 7.2}$	18.8$_{\uparrow 13.4}$	**53.7**$_{\uparrow 27.6}$	44.8$_{\uparrow 11.2}$	77.6$_{\uparrow 4.5}$	**91.8**$_{\uparrow 13.5}$
GA-Rollback	2.6$_{\uparrow 1.6}$	3.6$_{\uparrow 3.2}$	5.4$_{\uparrow 5.4}$	16.2$_{\uparrow 10.8}$	38.1$_{\uparrow 12.0}$	36.0$_{\uparrow 2.4}$	73.1	82.8$_{\uparrow 4.5}$
Single-Head Verifier	3.4$_{\uparrow 2.4}$	5.4$_{\uparrow 5.0}$	6.4$_{\uparrow 6.4}$	18.2$_{\uparrow 12.8}$	49.3$_{\uparrow 23.2}$	50.0$_{\uparrow 16.4}$	74.6$_{\uparrow 1.5}$	85.8$_{\uparrow 7.5}$
AURORA-Integration (Ours)	3.4$_{\uparrow 2.4}$	**6.6**$_{\uparrow 6.2}$	7.0$_{\uparrow 7.0}$	24.8$_{\uparrow 19.4}$	44.0$_{\uparrow 17.9}$	**55.2**$_{\uparrow 21.6}$	**80.6**$_{\uparrow 7.5}$	89.5$_{\uparrow 11.2}$
AURORA-Isolation (Ours)	1.4$_{\uparrow 0.4}$	4.8$_{\uparrow 4.4}$	4.2$_{\uparrow 4.2}$	**25.0**$_{\uparrow 19.6}$	45.7$_{\uparrow 19.6}$	48.2$_{\uparrow 14.6}$	70.1$_{\downarrow 3.0}$	90.3$_{\uparrow 12.0}$

suggest that AURORA is particularly effective under tight step constraints, where each rollback decision carries higher impact. **Obs.❷** *Dual-head verification of AURORA is most effective on medium-to-large scale models.* For LLaMA-3.1-8B and GLM4-9B, AURORA-Integration outperforms all baselines including Reflexion on both benchmarks. On ALFWorld, AURORA-Integration achieves 55.2% for LLaMA-3.1-8B, compared to Reflexion's 44.8%. For Qwen2.5-14B, AURORA-Integration surpasses Reflexion by 6 percentage points on Game of 24, improving the success rate from 18.8% to 24.8%, suggesting that step-level dual-head verification is particularly effective for tasks requiring precise multi-step reasoning. **Obs.❸** *Verification effectiveness of AURORA scales with model capacity.* On Qwen2.5-3B, Reflexion outperforms AURORA-Integration on ALFWorld, achieving 53.7% while AURORA-Integration only reaches 44.0%. In contrast, for models at 8B scale and above, this relationship inverts and AURORA-Integration generally outperforms Reflexion on both benchmarks. This because when the generator is weak, trajectory quality is limited and the verification signal becomes unreliable, making multi-trial retry a more robust strategy. As model scale increases, the learned verification becomes more reliable and AURORA-Integration's advantage over Reflexion becomes consistent.

C. Ablation Studies

We ablate two design choices in AURORA: (1) whether to include a global verification head alongside the local head, and (2) the context isolation strategy between the dual-heads. Results are reported in Table I, and we compare the following four configurations. GA-Rollback contains no verifier. Single-Head Verifier retains only the local validity head, which assesses only whether the current action is physical validity. AURORA-Integration and AURORA-Isolation both employ the full dual-head design described in Section III-D, differing only in whether context is shared at the attention-weight level or separated at the input level. And we make the following observations. **Obs.❹** *Dual-head design of AURORA outperforms single-head verification on medium-to-large models.* On ALFWorld, AURORA-Integration outperforms Single-Head on LLaMA-3.1-8B, improving from 50.0% to 55.2%, and on GLM4-9B, improving from 74.6% to 80.6%. On Qwen2.5-14B, both dual-head variants surpass single-head on both benchmarks. For Qwen2.5-3B, however,

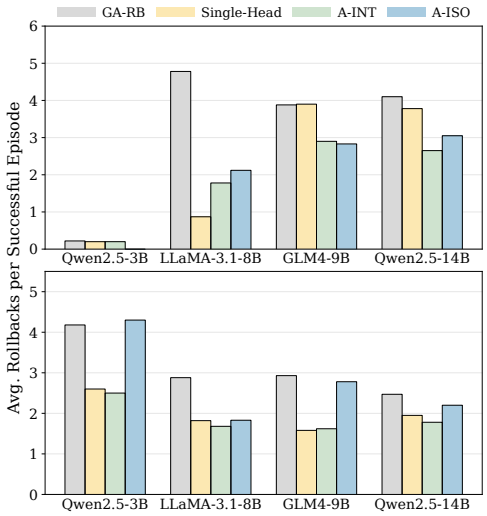


Fig. 5. **Average rollback count per successful episode on Game of 24 (top) and ALFWorld (bottom).** GA-RB: GA-Rollback; S-Head: Single-Head Verifier; A-INT: AURORA-Integration; A-ISO: AURORA-Isolation. Lower values indicate more selective assistant invocation.

single-head achieves 49.3% on ALFWorld while AURORA-Integration only reaches 44.0%, indicating that the global head's contribution diminishes when the generator cannot produce reliable trajectory-level signals. **Obs.❺** *Context isolation strategy of AURORA has a capacity-dependent effect.* For models at 3B, 8B, and 9B scale, AURORA-Integration generally outperforms AURORA-Isolation. On GLM4-9B, Integration achieves 80.6% on ALFWorld while Isolation drops to 70.1%. In contrast, for Qwen2.5-14B, AURORA-Isolation outperforms all rollback-based methods on both benchmarks, reaching 25.0% on Game of 24 and 90.3% on ALFWorld, competitive with the multi-trial Reflexion (91.8%) at significantly lower computational cost. This pattern reflects the difference in attention masking position: Integration applies masking at the attention-weight level and retains shared context across both heads, benefiting smaller models. Isolation applies masking at the input level and enforces strict context separation, which larger models can better exploit to extract independent physical validity and overall coherence signals.

D. Efficiency Analysis

We complement the ablation with two efficiency-focused analyses: the rollback frequency and path length. **Obs.❻**

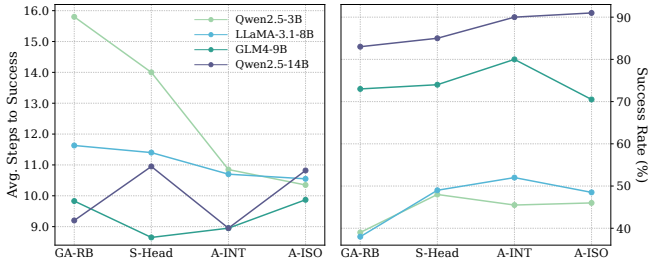


Fig. 6. **Average steps to success (left) and success rate (right) on ALFWorld across model scales.** GA-RB: GA-Rollback; S-Head: Single-Head Verifier; A-INT: AURORA-Integration; A-ISO: AURORA-Isolation. Lower steps with higher success rate indicates more efficient task completion.

AURORA-Integration reduces rollback frequency more than AURORA-Isolation. As shown in Figure 5, AURORA-Integration achieves the lowest or near-lowest rollback count in 7 of 8 model-dataset combinations. In contrast, AURORA-Isolation on Qwen2.5-3B (ALFWorld) produces 4.3 rollbacks per episode, exceeding the GA-Rollback, indicating that strict input separation prevents small models from generating reliable independent verification signals and leads to over-triggering. This suggests that shared attention between local and global heads in AURORA-Integration provides a more robust gating signal when model capacity is limited.

Obs.❼ *AURORA-Integration achieves simultaneous gains in path efficiency and success rate on medium-scale models.* Figure 6 shows that AURORA-Integration reduces average steps from 11.63 to 10.68 while raising success rate from 36.0% to 55.2% for LLaMA-3.1-8B; the same trend holds for GLM4-9B, where steps decrease from 9.83 to 8.98 and success rate improves from 73.1% to 80.6%. These gains reflect not only more accurate rollback decisions but also more targeted recovery paths. For Qwen2.5-14B, AURORA-Integration and Isolation show complementary strengths: AURORA-Integration achieves the shortest average path (8.93 steps) while AURORA-Isolation reaches the highest success rate 90.3%, suggesting that at larger model scale, strict context separation enables more precise per-rollback corrections at the cost of slightly longer paths.

E. Case Study

Figure 4 compares three methods on an ALFWorld cooling task. ReAct misidentifies the mug's location and falls into aimless exploration, exhausting its step budget without completing the task. GA-Rollback invokes the assistant unconditionally, which incorrectly instructs the agent to place the mug before cooling it, introducing new errors rather than correcting existing ones. AURORA intervenes only when verification signals raise an alarm. When the agent attempts to place the mug directly in the fridge, the local score drops sharply ($v_{\text{local}} = 0.023$, $v_{\text{global}} = 0.906$), indicating a physical execution error while the trajectory direction remains correct; the assistant applies targeted local correction. Later, a drop in global score ($v_{\text{global}} = 0.142$) triggers re-alignment of the overall trajectory. AURORA completes the task with three precisely timed interventions, in contrast to ReAct's zero and GA-Rollback's excessive corrections.

V. Conclusions

We propose AURORA, a dual-head attention-based verification framework for selective and precise rollback in LLM agents, addressing the error accumulation and redundant intervention caused by unconditional rollback strategies. AURORA derives two complementary verification scores from attention-masked interaction trajectories, capturing physical validity and overall coherence, and triggers rollback only when these signals indicate deviations. Experiments on Game of 24 and ALFWorld show that AURORA outperforms unverified baselines and single-head verification across model scales, achieving up to 19.2 percentage points improvement in success rate while reducing unnecessary rollbacks. As a plug-and-play module, AURORA integrates into existing rollback frameworks without modifying the generator or assistant. Future work will extend AURORA to simulated robotic manipulation and real-world navigation tasks, and explore human-in-the-loop annotation to mitigate potential biases from the closed-source labeling pipeline.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[2] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, "ALFWorld: Aligning text and embodied environments for interactive learning," in *ICLR*, 2021.

[3] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, T. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *CoRL*, 2023.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *ICLR*, 2023.

[5] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *NeurIPS*, 2023.

[6] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *ICLR*, 2024.

[7] X. Li, K. Chen, Y. Long, X. Bai, Y. Xu, and M. Zhang, "Generator-Assistant stepwise rollback framework for large language model agent," in *EMNLP*, 2025.

[8] A. Cuadron, P. Yu, Y. Liu, and A. Gupta, "Saber: Small actions, big errors—safeguarding mutating steps in llm agents," *arXiv preprint arXiv:2512.07850*, 2025.

[9] X. Feng, Z.-Y. Chen, Y. Qin, Y. Lin, X. Chen, Z. Liu, and J.-R. Wen, "Large language model-based human-agent collaboration for complex task solving," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, pp. 1336–1357.

[10] H. Wang, C. M. Poskitt, J. Wei, and J. Sun, "Probguard: Probabilistic runtime monitoring for llm agent safety," *arXiv preprint arXiv:2508.00500*, 2025.

[11] L. Chen, K. Lu, A. Rajeswaran, K. Lee, A. Grover, M. Laskin, P. Abbeel, A. Srinivas, and I. Mordatch, "Decision transformer: Reinforcement learning via sequence modeling," in *Advances in Neural Information Processing Systems*, 2021, pp. 15 084–15 097.

[12] M. Janner, Q. Li, and S. Levine, "Offline reinforcement learning as one big sequence modeling problem," in *Advances in Neural Information Processing Systems*, 2021.

[13] M. Ahn *et al.*, "Do as I can, not as I say: Grounding language in robotic affordances," in *CoRL*, 2022.

[14] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning, acting, and planning in language models," in *ICML*, 2024.

[15] J. Y. Koh, S. McAleer, D. Fried, and R. Salakhutdinov, "Tree search for language model agents," *arXiv preprint arXiv:2407.01476*, 2024.

[16] H. Lee, J. Kim, J. Willette, and S. J. Hwang, "SEA: Sparse linear attention with estimated attention mask," in *International Conference on Learning Representations*, 2024.

[17] H. Zhang, H. Fan, K. Sha, Y. Huang, and Y. Feng, "DAM: Dynamic attention mask for long-context large language model inference acceleration," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 4663–4676.

[18] X. Wu, A. Ajorlou, Y. Wang, S. Jegelka, and A. Jadbabaie, "On the role of attention masks and layernorm in transformers," in *Advances in Neural Information Processing Systems*, 2024, pp. 14 774–14 809.

[19] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," *arXiv preprint arXiv:2309.07864*, 2023.

[20] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, 2024.

[21] A. Costarelli, M. Allen, R. Hauksson, G. Sodunke, S. Hariharan, C. Cheng, W. Li, J. Clymer, and A. Yadav, "Gamebench: Evaluating strategic reasoning abilities of LLM agents," *arXiv preprint arXiv:2406.06613*, 2024.

[22] Z. Pan, J. Luo, Z. Xiao, K. Ding, X. Luo, and M. Zhang, "Zipact: Zipping interaction history into a compact state for efficient llm agents," *Transactions on Machine Learning Research*, 2026.

[23] Y. Zhang, S. Yang, C. Bai, F. Wu, X. Li, Z. Wang, and X. Li, "Towards efficient llm grounding for embodied multi-agent collaboration," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 1663–1699.

[24] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *NeurIPS*, 2022.

[25] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *NeurIPS*, 2024.

[26] M. Rizvi-Martel, S. Bhattamishra, N. Rathi, G. Rabusseau, and M. Hahn, "Benefits and limitations of communication in multi-agent reasoning," *arXiv preprint arXiv:2510.13903*, 2025.

[27] Y. Wang, L. Yang, G. Li, M. Wang, and B. Aragam, "ScoreFlow: Mastering LLM agent workflows via score-based preference optimization," *arXiv preprint arXiv:2502.04306*, 2025.

[28] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *Association for Computing Machinery*, 2023.

[29] D. Gosmar and D. A. Dahl, "Hallucination mitigation using agentic AI natural language-based frameworks," *arXiv preprint arXiv:2501.13946*, 2025.

[30] X. Liu, X. Yang, Z. Li, P. Li, and R. He, "AgentHallu: Benchmarking automated hallucination attribution of LLM-based agents," *arXiv preprint arXiv:2601.06818*, 2026.

[31] Y. Bai *et al.*, "Constitutional AI: Harmlessness from AI feedback," *arXiv preprint arXiv:2212.08073*, 2022.

[32] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *NeurIPS*, 2023.

[33] L. Pan, M. Saxon, W. Xu, D. Nathani, X. Wang, and W. Y. Wang, "Automatically correcting large language models: Surveying the landscape of diverse self-correction strategies," *Transactions on Machine Learning Research*, 2024, survey Track.

[34] R. Aksitov, S. Miryoosefi, Z. Li, D. Li, S. Babayan, K. Kopparapu, Z. Fisher, R. Guo, S. Prakash, P. Srinivasan, M. Zaheer, F. Yu, and S. Kumar, "ReST meets ReAct: Self-improvement for multi-step reasoning LLM agent," *arXiv preprint arXiv:2312.10003*, 2023.

[35] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and alignment," *arXiv preprint arXiv:2312.13010*, 2024.

[36] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *ICLR*, 2024.

[37] N. Z. J. H. Ryo Kamoi, Yusen Zhang and R. Zhang, "When can LLMs actually correct their mistakes? A critical survey of self-correction of LLMs," *Transactions of the Association for Computational Linguistics*, 2024.

[38] Y. Qu, T. Zhang, N. Garg, and A. Kumar, "Recursive introspection: Teaching language model agents how to self-improve," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 55 249–55 285.

[39] A. Kumar, V. Zhuang, R. Agarwal, Y. Su, J. D. Co-Reyes, A. Singh, K. Baumli, S. Iqbal, C. Bishop, R. Roelofs, L. M. Zhang, K. McKinney, D. Shrivastava, C. Paduraru, G. Tucker, D. Precup, F. Behbahani, and A. Faust, "Training language models to self-correct via reinforcement learning," *arXiv preprint arXiv:2409.12917*, 2024.

[40] T. Wang, P. Yu, X. E. Tan, S. O'Brien, R. Pasunuru, J. Dwivedi-Yu, O. Golovneva, L. Zettlemoyer, M. Fazel-Zarandi, and A. Celikyilmaz, "Shepherd: A critic for language model generation," *arXiv preprint arXiv:2308.04592*, 2023.

[41] S. Kim, J. Shin, Y. Cho, J. Jang, S. Longpre, H. Lee, S. Yun, S. Shin, S. Kim, J. Thorne, and M. Seo, "Prometheus: Inducing fine-grained evaluation capability in language models," in *The Twelfth International Conference on Learning Representations*, 2024.

[42] T. Hoang, K.-H. Huang, S. Kokane, J. Zhang, Z. Liu, M. Zhu, J. Grigsby, T. Lan, M. S. Ryoo, C.-S. Wu *et al.*, "LAM SIMULATOR: Advancing data generation for large action model training via online exploration and trajectory feedback," in *Findings of the Association for Computational Linguistics: ACL 2025*. Association for Computational Linguistics, 2025, pp. 12 921–12 934.

[43] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 31 997–32 016.

[44] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: Llm agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642.

[45] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, and Z. Hu, "Reasoning with language model is planning with world model," in *EMNLP*, 2023.

[46] R. Koohestani, "AgentGuard: Runtime verification of AI agents," in *Proc. 40th IEEE/ACM Int. Conf. Autom. Softw. Eng. Workshops (ASEW)*, 2025, pp. 74–77.

[47] J. Lu, Z. Dou, H. Wang, Z. Cao, J. Dai, Y. Feng, and Z. Guo, "AutoPSV: Automated process-supervised verifier," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 79 935–79 962.

[48] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[49] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *ICLR*, 2024.

[50] J. Uesato, N. Kushman, R. Kumar, F. Song, N. Siegel, L. Wang, A. Creswell, G. Irving, and I. Higgins, "Solving math word problems with process- and outcome-based feedback," *arXiv preprint arXiv:2211.14275*, 2022.

[51] J. Han, W. Buntine, and E. Shareghi, "VerifiAgent: A unified verification agent in language model reasoning," in *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025, pp. 16 410–16 431.

[52] X. Tang, T. Qin, T. Peng, Z. Zhou, D. Shao, T. Du, X. Wei, P. Xia, F. Wu, H. Zhu *et al.*, "Agent KB: Leveraging cross-domain experience for agentic problem solving," *arXiv preprint arXiv:2507.06229*, 2025.

[53] J. Shi, Y. Wu, B. Wu, Y. Peng, L. Wang, G. Liu, and Y. Luo, "Trainable dynamic mask sparse attention," *arXiv preprint arXiv:2508.02124*, 2025.

[54] Q. Team, "Qwen2. 5 technical report," *arXiv preprint arXiv:2412.13663*, 2024.

[55] A. . M. Llama Team, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[56] T. GLM, "Glm-4: An open language model family," *arXiv preprint arXiv:2406.12793*, 2024.

[57] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with disentangled attention," in *International Conference on Learning Representations*, 2021.

[58] DeepSeek-AI, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[59] F. Gilardi, M. Alizadeh, and M. Kubli, "Chatgpt outperforms crowd workers for text-annotation tasks," *Proceedings of the National Academy of Sciences*, p. e2305016120, 2023.